# Monte Carlo Sprint Forecasting

How the maths works — spreadsheet and web app — and how to explain it

Study guide · Digital2DNA · September 2026

**KEY POINT**

Everything in both tools reduces to three lines:

```
M = sprints × average per sprint
S = √sprints × standard deviation per sprint
Replay 1,000 futures drawn from Normal(M, S); count how many reach the target.
```

# Contents


1. The idea in one page
2. The inputs
3. Step A — Describing the team's pattern
4. Step B — Sizing the goal
5. Step C — One replay
6. Step D — A thousand replays: probability and percentiles
7. The analytic shortcut (check any number by hand)
8. Two measures: velocity and flow
9. The spreadsheet, cell by cell
10. The web app
11. Spreadsheet vs web app
12. Assumptions and limits — and how to answer them
13. Worked example 1: the app's sample data
14. Worked example 2: a real 37-sprint project
15. Meeting Q&A
16. One-page cheat sheet


# 1. The idea in one page

**The deterministic way** most teams plan: velocity is 40 points per sprint, we have 8 sprints, so we'll deliver 320. That single number is the *most likely* outcome, but it is wrong about half the time — the team will come in under 320 in roughly one future out of two. Committing to it is a coin toss dressed up as a plan.

**The probabilistic way:** keep the average, but also keep the *swing* — how much a sprint typically differs from the average. Then instead of one future, look at many. A Monte Carlo simulation is just that: replay the remaining sprints over and over, each time drawing every sprint at random from the team's real pattern, add them up, and keep the total. After 1,000 replays you have 1,000 plausible futures. Now you can answer the questions that matter:

- **What are the odds of hitting 300?** — the share of replays that reached 300.
- **What can we safely commit to?** — the amount reached in 85% of replays (P85).
- **How much room do we have?** — the gap between P85 and the target.

**SAY IT LIKE THIS**

We look at what the team has actually finished sprint after sprint — the average and how much it swings. Then we replay the rest of the project a thousand times using that pattern. The percentage is simply how many of those thousand futures got there. Instead of one number that's right half the time, you get odds — and a number you can commit to at 85%.

"Monte Carlo" is the name for any method that answers a question by running many random trials and counting outcomes. It was named after the casino. Nothing in it is exotic; it is a way of doing statistics by brute force so the result is visible as a pile of outcomes rather than a formula.

## 2. The inputs

| Input | Meaning | Where |
|---|---|---|
| Sprint history | For each past sprint: story points completed and stories completed. | Sheet rows 2–6 · App step 1 |
| History window | How many of the most recent sprints feed the statistics. | Sheet: the range in the formulas · App: Options, "all" or last N |
| Start date, target date | The period being forecast. | Sheet B13, B14 · App step 2 |
| Sprint length | Days per sprint; sets how many sprints fit in the period. | Sheet: fixed 7 in the formula · App: 1/2/3 weeks |
| Target story points | The scope, in points. | Sheet B15 · App step 2 |
| Story size (points per story) | Converts the point target into a story target. | Sheet B16 manual "Median Story Size" · App: from history, or manual |
| Trials | Number of replays. | Sheet: 1,000 in the formula · App: 1,000 / 5,000 / 10,000 |

**WATCH OUT**

Sprint length must match the history. If the rows are 2-week sprints and you tell the tool sprints are 1 week, it will count twice as many sprints and roughly double the forecast. Sprint length is not a speed dial; it is "how many of those past-looking sprints fit before the date".

## 3. Step A — Describing the team's pattern

Two numbers summarise the history for each measure (points, stories): the **mean** and the **sample standard deviation**.

```
mean       μ = Σx / n
std dev    σ = √( Σ(x − μ)² / (n − 1) )        ← spreadsheet STDEV, app sampleStdDev
```

The **n − 1** matters. When you estimate spread from a small sample, dividing by n underestimates it (the sample mean is always a little closer to its own points than the true mean would be). Dividing by n − 1 corrects that. Both tools use n − 1, so their numbers agree with `STDEV` and with what any statistician expects.

### Worked: the app's sample data

| Sprint | Points x | x − 40 | (x − 40)² | Stories |
|---|---|---|---|---|
| 1 | 42 | +2 | 4 | 9 |
| 2 | 36 | −4 | 16 | 7 |
| 3 | 45 | +5 | 25 | 10 |
| 4 | 38 | −2 | 4 | 8 |
| 5 | 41 | +1 | 1 | 8 |
| 6 | 39 | −1 | 1 | 9 |
| 7 | 44 | +4 | 16 | 8 |
| 8 | 35 | −5 | 25 | 7 |
| **Total** | **320** | **0** | **92** | **66** |

```
points   μ = 320 / 8 = 40.0        σ = √(92 / 7) = √13.14 = 3.63
stories  μ = 66 / 8  = 8.25        σ = 1.04   (same method)
```

Read σ as "a typical sprint lands within about 3.6 points of 40". About two sprints in three fall inside μ ± σ; about 19 in 20 inside μ ± 2σ.

### The history window

Older sprints may describe a team that no longer exists (people changed, the work changed). Using only the last N sprints trades a rougher estimate of σ for a more current one. Rule of thumb: use everything unless something real changed; never go below 5 if you can help it.

## 4. Step B — Sizing the goal

```
sprints available   n  = ⌊ (target date − start date) / sprint days ⌋      ← rounded DOWN
story size          s  = Σpoints / Σstories   (app "from history")   or a typed value
target stories      Tₛ = ⌊ target points / s ⌋                          ← rounded DOWN
```

Rounding down is deliberate: a sprint that ends after the date cannot help, and a fraction of a story is not a delivered story. The sheet divides by 7 (weekly sprints hard-coded); the app divides by 7, 14 or 21.

Worked: 4 Mar → 24 Jun 2026 is 112 days; ÷ 14 = 8 sprints. Story size 320 ÷ 66 = 4.85 points per story; 300 ÷ 4.85 = 61.9 → **61 stories**.

The original spreadsheet asks for a "Median Story Size" typed by hand (5 in the sample). The later version of the sheet computed it as total points ÷ total stories. The app does the latter by default because it is grounded in the same history

as everything else; the manual override exists for a project whose stories will be sized differently from the past.

## 5. Step C — One replay

A replay is the sum of the n remaining sprints, each drawn at random from the team's pattern. Two facts about adding independent random quantities give the whole model:

```
mean of the sum       M = n · μ
variance of the sum   = n · σ²      →   std dev of the sum   S = √n · σ
```

**KEY POINT**

Means add. Variances add. So the standard deviation grows with the *square root* of the number of sprints, not linearly. Over 8 sprints with σ = 3.63, the swing of the total is √8 × 3.63 = 10.3 points — not 8 × 3.63 = 29. Good and bad sprints partly cancel. This is why a longer horizon is *relatively* more predictable than a single sprint.

Worked: M = 8 × 40 = **320**; S = √8 × 3.63 = 2.83 × 3.63 = **10.25**.

### Why a normal (bell) distribution?

Individual sprints need not be bell-shaped. But the *sum* of several independent sprints is approximately bell-shaped whatever the individual shape — that is the Central Limit Theorem, and it kicks in usefully from about 5–6 sprints. So a replay is modelled as one draw from Normal(M, S) rather than as n separate draws added up. The two are mathematically identical for normal sprints, and one draw is 8× cheaper.

### How a random normal number is produced

**Spreadsheet:** `NORM.INV(RAND(), M, S)`. `RAND()` gives a uniform number between 0 and 1 — think of it as a random percentile. `NORM.INV` is the inverse cumulative distribution: it answers "which value of a Normal(M, S) sits at that percentile?" Feed it 0.5 and you get M; feed it 0.15 and you get M − 1.036 S; feed it a random percentile and you get a random normal value. This is called *inverse-transform sampling*.

**Web app:** the Box–Muller transform. Take two uniforms $u_1$, $u_2$ and compute

```
z = √(−2 · ln u₁) · cos(2π · u₂)        → z is a standard normal (mean 0, sd 1)
x = max(0, M + S · z)                    → scaled, shifted, floored at zero
```

Same distribution, different recipe (JavaScript has no built-in inverse normal). The `max(0, …)` floor stops a replay delivering negative work; it never matters in practice because a target is never near zero, but it keeps the histogram honest.

## 6. Step D — A thousand replays

Repeat Step C 1,000 times (the app also offers 5,000 and 10,000). Sort the results. Everything the tools report is read off this sorted pile.

### Probability of hitting the target

```
P = (number of replays ≥ target) / (number of replays)
sheet:  =COUNTIF(pile, ">=" & target) / 1000
app:    fractionAtLeast(sorted, target)   — binary search for the first value ≥
target
```

### Percentiles: P50, P75, P85, P95

"P85" means *the amount you reach in 85% of replays*. That is the value with 85% of the pile *above* it — the **15th percentile** of the sorted pile, not the 85th. Hence:

| Label | Meaning | Percentile of the pile | Sheet / app call |
|---|---|---|---|
| P50 | a coin toss — half the futures do better, half worse | 50th | `PERCENTILE(pile, 0.50)` |
| P75 | probable | 25th | `PERCENTILE(pile, 0.25)` |
| P85 | safe commitment | 15th | `PERCENTILE(pile, 0.15)` |
| P95 | near certain | 5th | `PERCENTILE(pile, 0.05)` |

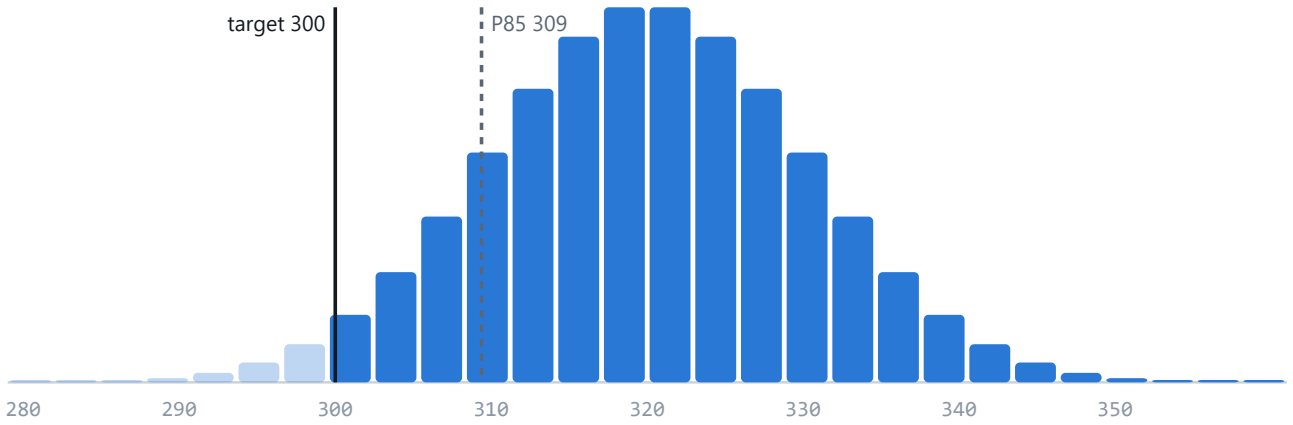
Both tools use the *inclusive* percentile with linear interpolation: rank = p × (N − 1); if that lands between two sorted values, interpolate. With 1,000 values, P85 is the value at rank 149.85 — i.e. 85% of the way from the 150th smallest to the 151st. The app copies the spreadsheet's rule exactly so the two agree.

**SAY IT LIKE THIS**

P85 is the number we'd hit in 85 of every 100 futures. If we commit to it, we're wrong about one time in seven — which is the level most stakeholders are comfortable with. P50 is the number you'd be wrong about half the time; that's what velocity-times-sprints gives you.

### The histogram

The pile is bucketed into ~28 bins and each bin's count drawn as a bar. The solid line is the target; the dashed line is P85. Dark bars are replays that hit the target, pale bars fell short — so the probability is literally dark bars ÷ all bars. The bell shape is not drawn; it emerges from the replays.




The pile of 1,000 replays for the sample data (M = 320, S = 10.25). Solid line: target 300. Dashed: P85 ≈ 309. Pale bars fall short of the target — about 2.5% of the area.

**Why the numbers wobble when you re-roll.** A percentage estimated from N random replays has a standard error of $\sqrt{p(1-p)/N}$. At p = 0.975 that is ±0.5 percentage points with 1,000 replays and ±0.16 with 10,000. The bell's position and width don't change between rolls; only the bar heights jiggle. Treat the last digit as noise and the range as the message.

## 7. The analytic shortcut

Because each replay is a normal draw, everything the simulation produces can also be computed with a formula — which is the best sanity check you can do in front of someone. With M = nμ and S = √n σ, and z-multipliers from the standard normal table:

| Quantity | Formula | z-multiplier |
|---|---|---|
| Probability of reaching T | 1 − Φ( (T − M) / S ) | – |
| P50 | M | 0 |
| P75 | M − 0.674 · S | 0.674 |
| P85 | M − 1.036 · S | 1.036 |
| P95 | M − 1.645 · S | 1.645 |

Φ is the standard normal cumulative function — in a spreadsheet, `NORM.S.DIST(z, TRUE)`. A short table of Φ for negative z (the cases that matter):

| z | -0.5 | -1.0 | -1.5 | -2.0 | -2.5 | -3.0 |
|---|---|---|---|---|---|---|
| P(reach target) = 1 − Φ(z) | 69% | 84% | 93% | 97.7% | 99.4% | 99.9% |

Worked: T = 300, M = 320, S = 10.25 → z = (300 − 320)/10.25 = **−1.95** → probability ≈ **97.4%**. P85 = 320 − 1.036 × 10.25 = **309.4**. The app reported 97.8% and 309 — agreement within dice noise.

**KEY POINT**

If someone asks "why simulate at all when a formula gives the answer?" — for this model you genuinely don't have to; the simulation is a numeric way of evaluating the normal distribution. Three reasons it is still the right engine: (1) the histogram shows people what "odds" means in a way a z-table never will; (2) the same engine keeps working if the model is later changed to something without a closed form — for example resampling actual sprints instead of assuming a bell; (3) it is the same method the industry tools use, so the results are comparable.

## 8. Two measures: velocity and flow

Points measure **effort**; stories measure **scope** (things actually finished). Both are simulated with identical maths from their own μ and σ. Why bother? Because a project isn't done until both land, and because comparing the two exposes a problem a single measure hides.

### Why the two probabilities can differ at all

When story size comes from history, s = Σpoints ÷ Σstories, so the story target is $T_s = T \div s = T \times (\Sigma\text{stories} \div \Sigma\text{points})$. The expected story total is $n \times \mu_s$, and $\mu_s \div \mu_p = \Sigma\text{stories} \div \Sigma\text{points}$ as well. Therefore

> `expected stories / story target` = $n \cdot \mu_s / (T \cdot \mu_s/\mu_p) = n \cdot \mu_p / T$ = `expected points / point target`

The two measures sit at *exactly* the same fraction of their targets on average. The only thing that can separate the two probabilities is **relative spread**: σ/μ (the coefficient of variation). In the sample data it is 9% for points and 12.5% for stories, so flow odds come out slightly lower. With a manual story size the means can differ too, because the planned scope is then sliced differently from the past.

### Reading a gap

| Pattern | What it means | What to do |
|---|---|---|
| Point odds high, story odds low | Effort is fine; *finishing* is erratic. Story count per sprint swings more than points: inconsistent story sizes, or work that starts but doesn't finish. | Keep stories small and similar; limit work in progress; trim per-story overhead (reviews, hand-offs, deploys). |
| Story odds high, point odds low | A steady number of stories closes, but effort per sprint swings — a few large stories carry most of the points. | Split the big ones. |
| Both within ~15 points of each other | Effort and scope agree; the forecast isn't hiding a sizing problem. | Commit on the numbers. |

**The golden rule:** commit on whichever P85 is *lower relative to its target*. Both effort and scope must be delivered, so the tighter one is the honest commitment. The app computes P85 ÷ target for each measure and names the smaller.

**WATCH OUT**

The original spreadsheet's notes said "stories are too small — merge trivial stories" for the first pattern. That contradicts the point of small stories and isn't what the maths says. A gap is a *consistency* signal, never a "make them bigger" signal.

## 9. The spreadsheet, cell by cell

The Apps Script builds one sheet with three columns. Column A is labels, B is points / velocity, C is stories / flow.

| Row | A (label) | B (points / velocity) | C (stories / flow) |
|---|---|---|---|
| 1 | SPRINT HISTORY | Story Points | Stories Completed |
| 2–6 | Sprint 1 … Sprint 5 | values | values |
| 9 | Rolling Average | `=AVERAGE(B2:B6)` | `=AVERAGE(C2:C6)` |
| 10 | Standard Deviation | `=STDEV(B2:B6)` | `=STDEV(C2:C6)` |
| 13 | Start Date | date | |
| 14 | Target Date | date | |
| 15 | Target Story Points | number | |
| 16 | Median Story Size | number (typed) | |
| 17 | Target Scope (Stories) | `=INT(B15/B16)` | |
| 18 | Sprints Available | `=MAX(0, INT((B14-B13)/7))` | |
| 21 | Probability of Hitting Targets | `=IF(B18<=0, 0, COUNTIF(MAKEARRAY(1000,1, LAMBDA(r,c, NORM.INV(RAND(), B18*B9, SQRT(B18)*B10))), ">="&B15) / 1000)` | same with C9, C10, B17 |
| 24 | P50 | `=IF(B18<=0, 0, PERCENTILE(MAKEARRAY(1000,1, LAMBDA(r,c, NORM.INV(RAND(), B18*B9, SQRT(B18)*B10))), 0.50))` | same |
| 25 | P75 | … `0.25` ) | same |
| 26 | P85 | … `0.15` ) | same |
| 27 | P95 | … `0.05` ) | same |

### Reading the long formula from the inside out

```
NORM.INV(RAND(), B18*B9, SQRT(B18)*B10)
   one replay: a random normal with mean n·μ (B18×B9) and sd √n·σ (SQRT(B18)×B10)

LAMBDA(r, c, …)
   wraps that expression as a function of row and column (both ignored)

MAKEARRAY(1000, 1, …)
   calls it 1,000 times → a column of 1,000 replays (the pile)

COUNTIF(pile, ">=" & B15) / 1000
   share of replays at or above the target             → probability

PERCENTILE(pile, 0.15)
   inclusive percentile of the pile                     → P85

IF(B18<=0, 0, …)
   guard: no sprints between the dates → report 0 instead of an error
```

### Things to know about the sheet

- **Every result cell rolls its own pile.** B21, B24, B25, B26, B27 each call `MAKEARRAY` separately, so the probability and the four percentiles come from five *different* sets of 1,000 replays. They are consistent within noise but not from the same draw. The app uses one pile for everything.
- **Recalculation.** `RAND()` re-rolls whenever the sheet recalculates — any edit does it. The menu item "Recalculate Odds" writes a timestamp into D1 purely to force that recalculation.
- **Changing history length.** The formulas reference B2:B6 (5 sprints). To use more, insert rows and extend every range (B9, B10, C9, C10) — or point them at a wider range from the start.
- **Changing trials.** Edit the 1000 in `MAKEARRAY(1000, …)` and the divisor in the COUNTIF cell. 1,000 is plenty; 10,000 makes the sheet noticeably slower because five cells each build 10,000 values on every recalculation.
- **Sprint length.** Hard-coded weekly ( `/7` ). Change to `/14` for fortnightly sprints.
- **Negative replays.** Not floored. Irrelevant unless M is within ~3S of zero.
- **Excel.** `MAKEARRAY` , `LAMBDA` , `NORM.INV` , `PERCENTILE` all exist in Excel 365. Older Excel lacks MAKEARRAY/LAMBDA: the fallback is a helper column of 1,000 rows each containing `=NORM.INV(RAND(), $B$18*$B$9, SQRT($B$18)*$B$10)` and pointing COUNTIF/PERCENTILE at it.
- **Apps Script** only builds and formats the sheet; no simulation runs in the script. Once built, the sheet is self-contained and the script can be deleted.

## 10. The web app

Same model, packaged so nobody needs to touch a formula. Plain TypeScript, no framework, no server: the page is static files (Vite build), data stays in the browser.

| Where | What |
|---|---|
| `domain/statistics.ts` | mean, sample std dev (n − 1), inclusive percentile, fraction ≥ threshold |
| `domain/random.ts` | Box–Muller standard normal; a seeded generator for tests |
| `domain/history.ts` | history window, μ and σ per measure, sprints available, story size, story target |
| `domain/simulation.ts` | `simulateDelivery` : N draws from Normal(nμ, √nσ), floored at 0, sorted · `runForecast` : both measures |
| `domain/interpretation.ts` | confidence bands (85% / 50%), the gap diagnosis, the golden rule |
| `services/` | localStorage save, share links (data in the URL fragment), JSON export/import, spreadsheet paste parser |
| `ui/` | three steps + the answer; breakdown on demand; "?" glossary; histograms |
| `tests/` | 46 unit tests, including checks that the maths reproduces the spreadsheet's figures |

**Persistence and sharing.** Inputs are saved in the browser that typed them (per device, never uploaded). To hand a forecast to someone: *Copy share link* encodes the whole dataset in the URL, or *Export*/*Import* a JSON file. A shared, always-current team dataset would need a small backend; the storage layer is written so that is a one-file change.

**Verdict bands.** ≥ 85% "Safe to commit" · 50–85% "Likely, not safe" · < 50% "Unlikely". These thresholds are conventions, not laws; 85% is the widely used commitment level, 50% is where a coin toss starts.

## 11. Spreadsheet vs web app

| | Spreadsheet | Web app |
|---|---|---|
| Model | Identical: Normal(nμ, √nσ), N replays, count ≥ target, inclusive percentiles | |
| History window | Fixed range (B2:B6); extend by editing formulas | Any number of rows; all or last N |
| Story size | Typed ("Median Story Size") | From history (Σpts ÷ Σstories) or typed |
| Sprint length | 7 days hard-coded | 1 / 2 / 3 weeks |
| Trials | 1,000 (edit the formula) | 1,000 / 5,000 / 10,000 |
| Random normal | `NORM.INV(RAND())` | Box–Muller |
| Negative replays | Possible (never matters) | Floored at 0 |
| Piles | One per result cell (five per measure) | One per measure |
| Re-roll | Any edit / timestamp menu | Button |
| Interpretation | Static notes | Computed: which pattern, which P85 binds |
| Histogram | None | Yes, both measures |
| Explanations | None | "?" on each concept, "how it works" panel |
| Sharing | Share the sheet | Share link / JSON file |
| Runs in | Google Sheets, Excel 365 | Any browser; Vercel-hosted |
| Auditability | Every formula visible in the cell | Open source, unit-tested |
| Best for | Someone who lives in spreadsheets and wants to tweak | A team that wants an answer without learning the model |

## 12. Assumptions and limits — and how to answer them

| Assumption | What it means | Honest answer / mitigation |
|---|---|---|
| Stationarity — the future looks like the past | Same team, same kind of work, same definition of done, same estimation habits. | The forecast is only as good as the history's relevance. Use the window to drop stale sprints; re-run every sprint review; note known changes (new hire, holiday season, rewrite) as caveats. |
| Independence between sprints | A bad sprint doesn't make the next one bad. | Real teams have streaks (incidents, leave), which makes the true spread slightly wider than modelled — the forecast is a touch optimistic. Longer history captures more of that. |
| Normal total | The sum of sprints is bell-shaped. | Solid from ~5–6 sprints (Central Limit Theorem). Shaky with 2–3. The app warns under 3. |
| σ estimated from few sprints | Five sprints give a spread estimate that can itself be off by ~30%. | More history helps more than more trials. 8–12 sprints is a comfortable base. |
| Fixed scope | The target doesn't grow. | Not modelled. When scope changes, change the target and re-run; the forecast is a snapshot. |
| Points are comparable over time | A 5 today is a 5 last quarter. | Re-estimation, inflation or a new scale breaks the pattern. Stories per sprint is more robust to this — which is one reason both are tracked. |
| Whole sprints only | Partial sprints are dropped. | Conservative by design. |
| No capacity model | Holidays, on-call, team size changes aren't inputs. | Either accept the average captures them, or trim the window to sprints with comparable capacity. |

**KEY POINT**

The strongest single line: *the tool doesn't predict the future; it tells you what the past implies if nothing changes. Everything you know that the past doesn't is a caveat you add out loud.*

## 13. Worked example 1 — the app's sample data

| Step | Points (velocity) | Stories (flow) |
|---|---|---|
| History (8 sprints) | 42 36 45 38 41 39 44 35 | 9 7 10 8 8 9 8 7 |
| μ, σ | 40.00, 3.63 | 8.25, 1.04 |
| Goal | 300 points | ⌊300 / 4.85⌋ = 61 stories |
| Sprints n | ⌊112 / 14⌋ = 8 | |
| M = nμ | 320.0 | 66.0 |
| S = √n σ | 10.25 | 2.93 |
| z = (T − M)/S | −1.95 | −1.71 |
| Probability (analytic) | 97.4% | 95.6% |
| Probability (app, one roll) | 97.8% | 95.3% |
| P50 / P75 / P85 / P95 | 320 / 313 / 309 / 303 | 66 / 64 / 63 / 61 |
| P85 ÷ target | 1.031 | 1.033 |
| Verdict | Both "Safe to commit". Odds within 15 points → measures agree. Golden rule: points P85 ratio is marginally lower → commit on **309 points**. | |

## 14. Worked example 2 — a real 37-sprint project

From the spreadsheet version used in practice: 33 sprints of history, weekly sprints, 26 Feb → 18 Nov 2026, target 684 points.

| Step | Points | Stories |
|---|---|---|
| μ, σ (sheet rows 40–41) | 21.6, 6.4 | 6.7, 2.2 |
| Story size | 684 ÷ 212.7 → 3.22 points/story (from history) → target 212 stories | |
| Sprints n | ⌊265 / 7⌋ = 37 | |
| M = nμ | 799.2 | 247.9 |
| S = √n σ | 6.08 × 6.4 = 38.9 | 6.08 × 2.2 = 13.4 |
| z | (684 − 799)/38.9 = −2.96 | (212 − 248)/13.4 = −2.68 |
| Probability (analytic) | 99.85% | 99.6% |
| Probability (sheet) | 99.7% | 99.5% |
| P85 (analytic) | 799 − 1.036 × 38.9 = 759 | 248 − 1.036 × 13.4 = 234 |
| P85 (sheet) | 758 | 234 |
| P95 (analytic / sheet) | 735 / 736 | 226 / 227 |

Note how much wider this project's relative spread is (σ/μ ≈ 30% vs 9% in example 1) and yet the odds are excellent: 37 sprints of averaging (√37 ≈ 6) shrink the relative swing of the total to about 5%. Long horizons forgive noisy sprints.

# 15. Meeting Q&A

**What is a Monte Carlo simulation, in one sentence?**
A way of answering "what are the odds?" by replaying the situation many times with random variation and counting how often each outcome happens.

**Isn't this just velocity times sprints?**
Velocity × sprints is the middle of our bell — the P50. We keep that *and* the spread, so we can also say how likely the target is and what number is safe at 85%.

**Where does the randomness come from?**
From your own history. The team's sprint-to-sprint swing (the standard deviation) is what the dice are loaded with. No made-up numbers.

**Why 1,000 replays? Why not 100 or a million?**
1,000 puts the noise on a percentage at about ±0.5 points — below anything you'd act on. 10,000 is available and smooths the histogram. A million would change nothing visible.

**Why do the numbers change when I click again?**
New dice. The bell's position and width are fixed by your history; only the random pile jiggles. If 97% becomes 96%, nothing changed.

**Why is P85 the 15th percentile?**
"85% confidence" means 85% of futures do at least that well. 85% are above it, so 15% are below — it is the 15th percentile of the pile.

**Why does the spread only grow with √n?**
Good and bad sprints partly cancel. Variances add, standard deviations don't; $\sqrt{(n \cdot \sigma^2)} = \sqrt{n} \cdot \sigma$. Eight sprints at ±3.6 each give a total at ±10, not ±29.

**Why a normal distribution — sprints aren't bell-shaped.**
Individual sprints don't need to be. The *sum* of several independent sprints is close to normal regardless — the Central Limit Theorem. From about five sprints onwards it's a good fit.

**Could I get the same answers with a formula instead?**
Yes — for this model, $1 - \Phi((T - M)/S)$ and $M - 1.036 \cdot S$ give the probability and P85 directly. The simulation is the same maths done visibly, and it keeps working if we later change the model to one without a formula (e.g. resampling real sprints).

**What's the difference between the two percentages?**
Points measure effort; stories measure things actually finished. They're simulated separately. Because story size comes from the same history, their averages line up exactly; if the odds differ, it's because one measure swings more per sprint — usually inconsistent story sizes.

**Which number do I commit to?**

The P85 that is lower relative to its target. You need both effort and scope to land, so the tighter one is the honest commitment.

**How much history do I need?**

Three is the minimum for a spread to mean anything; five is workable; eight to twelve is comfortable. Beyond that, relevance matters more than quantity — drop sprints from before a real change.

**What if the team changed last month?**

Use the window to keep only sprints since the change, accept a rougher spread, and say so. The tool reflects the past; it can't know what the past doesn't contain.

**What about holidays and leave?**

If they're in the history, they're in the average and the spread. If a known dip is coming, either drop a sprint from n or treat the forecast as optimistic by that much.

**Does it handle scope creep?**

No — the target is fixed. Re-run with the new target when scope changes. Each run is a snapshot; keeping the share links gives you a history of forecasts.

**Can two people use the same data?**

Via share link or export/import today. A shared live dataset needs a small backend, which the code is structured to accept.

**Is my data uploaded anywhere?**

No. The app is static files; sprints live in your browser's storage. A share link carries the data inside the URL itself.

**Why did the odds jump when I changed sprint length?**

Sprint length changes how many sprints fit before the date. If your history rows are two-week sprints and you select one week, the tool counts twice as many sprints and doubles the forecast. Sprint length must match the rows.

**Spreadsheet or app?**

Same model. Spreadsheet if you want to see and tweak every formula; app if you want teams to get an answer without learning the model, plus the histogram and the plain-language reading.

**Can it be wrong?**

It can be confidently wrong if the future doesn't resemble the past. It is never a prediction — it's what the history implies. Say the caveats out loud; that's the job of the person presenting it.

# 16. One-page cheat sheet

### Describe the pattern

```
μ = Σx / n
σ = √( Σ(x−μ)² / (n−1) )
```

### Size the goal

```
n  = ⌊ days / sprint days ⌋
s  = Σpoints / Σstories
Tₛ = ⌊ T / s ⌋
```

### One replay

```
M = n·μ     S = √n·σ
x = max(0, NORM.INV(RAND(), M, S))
```

### The pile (N replays)

```
P(hit)  = count(x ≥ T) / N
P50     = PERCENTILE(pile, .50)
P75     = PERCENTILE(pile, .25)
P85     = PERCENTILE(pile, .15)
P95     = PERCENTILE(pile, .05)
```

### Analytic check

```
z = (T − M) / S
P(hit) = 1 − Φ(z)
P75 = M − 0.674·S
P85 = M − 1.036·S
P95 = M − 1.645·S
```

### Noise on a percentage

```
± √( p(1−p) / N )
p=.975: N=1000 → ±0.5 pts
        N=10000 → ±0.16 pts
```

### Verdicts

```
≥ 85%   Safe to commit
50–85%  Likely, not safe
< 50%   Unlikely
```

### Golden rule

```
commit on the measure with the
lower  P85 / target
```

## Glossary

| | |
|---|---|
| **Velocity** | Story points completed per sprint. |
| **Flow** | Stories completed per sprint (throughput). |
| **Standard deviation** | Typical distance of a sprint from the average. The "±". |
| **Sample std dev (n−1)** | The version used when the data is a sample; spreadsheet STDEV. |
| **Normal distribution** | The bell curve; fully described by a mean and a standard deviation. |
| **Central Limit Theorem** | Sums of independent random quantities tend to a normal shape. |
| **NORM.INV / inverse CDF** | Turns a percentile (0–1) into the value at that percentile of a normal. |
| **Box–Muller** | A formula turning two uniform randoms into a standard normal random. |
| **Percentile (inclusive)** | Value at rank p·(N−1) of the sorted pile, interpolated. |
| **P85** | Amount reached in 85% of replays = 15th percentile of the pile. |
| **Coefficient of variation** | σ / μ — spread relative to the average; what separates the two odds. |
| **Trial / replay** | One simulated future: one draw from Normal(M, S). |